Занятие2.2. Комментарии

Комментарий — это строка (или несколько строк) текста, которая вставляется в исходный код для объяснения того, что делает код. В языке C++ есть 2 типа комментариев: однострочные и многострочные.

Оглавление:

1. **Однострочные комментарии**
2. **Многострочные комментарии**
3. **Как правильно писать комментарии?**
4. **Закомментировать код**

Однострочные комментарии

Однострочные комментарии — это комментарии, которые пишутся после символов `//`. Они пишутся в отдельных строках и всё, что находится после этих символов комментирования, — игнорируется компилятором, например:

```
// std::cout << «Hello, world!» << std::endl; // всё, что находится справа от любого слеша, игнорируется компилятором
```

Как правило, однострочные комментарии используются для объяснения одной строчки кода:

```
std::cout << «Hello, world!» << std::endl; // cout и endl находятся в библиотеке iostream
std::cout << «It's so exciting!» << std::endl; // эти комментарии усложняют чтение кода
std::cout << «Yeah!» << std::endl; // особенно когда строки разной длины
```

Размещая комментарии справа от кода, мы затрудняем себе как чтение кода, так и чтение комментариев. Следовательно, однострочные комментарии лучше размещать над строками кода:

```
// cout и endl находятся в библиотеке iostream
std::cout << «Hello, world!» << std::endl;

// теперь уже легче читать:
std::cout << «It's so exciting!» << std::endl;

// но так ли?
std::cout << «Yeah!» << std::endl;
```

Многострочные комментарии

Многострочные комментарии — это комментарии, которые пишутся между символами `/* */`. Всё, что находится между звёздочками, — игнорируется компилятором:

```
/* Это многострочный комментарий.
Эта строка игнорируется
и эта тоже. */
```

Так как всё, что находится между звёздочками, — игнорируется, то иногда вы можете наблюдать следующее:

```
/* Это многострочный комментарий.
* Звёздочки слева
* упрощают чтение текста
*/
```

Многострочные комментарии не могут быть вложенными (т.е. одни комментарии внутри других):

```
/* Это многострочный /* комментарий */ а это уже не комментарий */
// Верхний комментарий заканчивается перед первым */, а не перед вторым */
```

Правило: Никогда не используйте вложенные комментарии.

Как правильно писать комментарии?

Во-первых, на уровне библиотек/программ/функций комментарии отвечают на вопрос **«ЧТО?»**: «Что делают эти библиотеки/программы/функции?». Например:

```
// Эта программа вычисляет оценку студента за семестр на основе его оценок за модули

// Эта функция использует метод Ньютона для вычисления корня функции

// Следующий код генерирует случайное число
```

Все эти комментарии позволяют понять, что делает программа, без необходимости смотреть на исходный код. Это особенно важно специалистам, работающим в команде, где не каждый специалист будет знаком со всем имеющимся кодом.

Во-вторых, внутри библиотек/программ/функций комментарии отвечают на вопрос **«КАК?»**: «Как код выполняет задание?». Например:

```
/* Для расчета итоговой оценки ученика, мы складываем все его оценки за уроки и домашние задания,
   а затем делим получившееся число на общее количество оценок.
   Таким образом, мы получаем средний балл ученика. */
```

Или:

```
// Чтобы получить рандомный (случайный) элемент, мы выполняем следующее:

// 1) Составляем список всех элементов.
// 2) Выполняем среднее значение для каждого элемента, исходя из его веса, цвета и цены.
// 3) Выбираем любое число.
// 4) Приравниваем соответствие элемента случайно выбранному числу.
// 5) Возвращаем случайный элемент.
```

Эти комментарии позволяют читателю понять, каким образом код выполняет поставленное ему задание.

В-третьих, на уровне стейтментов (однострочного кода) комментарии отвечают на вопрос **«ПОЧЕМУ?»**: «Почему код выполняет задание именно так, а не иначе?». Плохой комментарий на уровне стейтментов объясняет, что делает код. Если вы когда-нибудь писали код, который был настолько сложным, что нужен был комментарий, который бы объяснял, что он делает, то вам нужно было бы не писать комментарий, а переписывать этот код.

Примеры плохих и хороших однострочных комментариев:

Плохой комментарий:

```
// Присваиваем переменной sight значение 0
sight = 0;
```

(По коду это и так понятно)

Хороший комментарий:

```
// Глаза дракона залиты слепотой и ничего не видят
sight = 0;
```

(Теперь мы знаем, ПОЧЕМУ зрение у игрока равно нулю)

Плохой комментарий:

```
// Рассчитываем стоимость элементов
cost = items / 2 * storePrice;
```

(Да, мы видим, что здесь подсчет стоимости, но почему элементы делятся на 2?)

Хороший комментарий:

```
// Или нужно разделить все элементы на 2, потому что они куплены по парам
cost = items / 2 * storePrice;
```

(Теперь понятно!)

Программистам часто приходится принимать трудные решения по поводу того, каким способом решить проблему. А комментарии и существуют для того, чтобы напомнить себе (или объяснить другим) причину, почему вы написали код именно так, а не иначе.

Хорошие комментарии:

```
// Мы решили использовать сумму вместо массива,
// потому что массивы осуществляют медленную вставку.
```

Или:

```
// Мы используем метод Ньютона для вычисления корня функции,
// так как другого детерминистического способа решения этой задачи - нет.
```

И, наконец, комментарии нужно писать так, чтобы человек, который не имеет ни малейшего представления о том, что делает ваш код — смог в нем разобраться. Очень часто случаются ситуация, когда программист говорит: «Это же совершенно очевидно, что делает код! Я это точно не забуду!». Угадайте, что случится через несколько недель или даже дней? Это не совершенно очевидно, и вы удивитесь, как скоро вы забудете то, что делает ваш код. Вы (или кто-то другой) будете очень благодарны себе за то, что оставите комментарии, объясняя на человеческом языке что, как и почему делает ваш код. Читать отдельные строки кода — легко, понимать их логику и смысл — сложно.

Подытожим:

- На уровне библиотек/программ/функций оставляйте комментарии, отвечая на вопрос «*ЧТО?*».
- Внутри библиотек/программ/функций оставляйте комментарии, отвечая на вопрос «*КАК?*».
- На уровне стейтментов оставляйте комментарии, отвечая на вопрос «*ПОЧЕМУ?*».

Закомментировать код

Закомментировать код — это конвертировать одну или несколько строк кода в комментарии. Таким образом, вы можете (временно) исключить часть кода из компиляции.

Чтобы закомментировать одну строку кода, используйте однострочные символы комментирования `//`.

Не закомментировано:

```
std::cout << 1;
```

Закомментировано:

```
//    std::cout << 1;
```

Чтобы закомментировать блок кода, используйте однострочные символы комментирования `//` на каждой строке или символы многострочного комментария `/* */`.

Не закомментировано:

```
std::cout << 1;
std::cout << 2;
std::cout << 3;
```

Закомментировано символами однострочного комментария:

```
//    std::cout << 1;
//    std::cout << 2;
//    std::cout << 3;
```

Закомментировано символами многострочного комментария:

```
/*
    std::cout << 1;
    std::cout << 2;
    std::cout << 3;
*/
```

Есть несколько причин, почему следует использовать «закомментирование»:

- ***Причина №1***: Вы работаете над новой частью кода, которая пока что не рабочая, но вам нужно запустить программу. Компилятор не позволит выполнить программу, если в ней будут ошибки. Временное отделение нерабочего кода от рабочего комментированием позволит вам запустить программу. Когда код будет рабочий, то вы сможете его легко раскомментировать и продолжить работу.
- ***Причина №2***: Вы написали код, который компилируется, но работает не так, как нужно и сейчас у вас нет времени с этим возиться. Закомментируйте код, а затем, когда будет время, исправьте ошибки.
- ***Причина №3***: Поиск корня ошибки. Если вас не устраивают результаты работы программы (или вообще происходит сбой), полезно будет поочерёдно «отключать» части вашего кода, чтобы понять какие из них рабочие, а какие — создают проблемы. Если вы закомментируете одну или несколько строчек кода и программа начнет корректно работать (или пропадут сбои), шансы того, что последнее, что вы закомментировали, является ошибкой — очень велики. После этого вы сможете разобраться с тем, почему же этот код не работает так, как нужно.
- ***Причина №4***: Тестирование нового кода. Вместо удаления старого кода, вы можете его закомментировать и оставить для справки, пока не будете уверены в том, что ваш новый код работает так, как нужно. Как только вы будете уверены в новом коде, то сможете без проблем удалить старые фрагменты кода. Если же новый код у вас будет работать не так, как нужно, то вы сможете его удалить и откатиться к старому коду.

> ***Примечание:*** Во всех следующих уроках я буду использовать комментарии в иллюстративных целях. Внимательные читатели смогут заметить, что по вышеуказанным стандартам большинство из этих комментариев будут плохими. Но помните, что использовать я их буду в образовательных целях, а не для демонстрации хороших примеров.